MUGEN: Bubblun AI Patch / Calculating headbutt velocity slopes

X → abs(enemynear, pos x - pos x) <= ((enemynear, pos y - pos y) * 1.429) (due to the diagnol nature lol)

Keep this in mind.

Do the math for the following situation:
Var(11); keep this variable in mind.

Headbutt X | Var(11) = 1:
* velset x = 2.5
* velset y = -4

Headbutt Y | Var(11) = 2:
* velset x = 4
* velset y = -6.5

Headbutt Z | Var(11) = 3:
* velset x = 6
* velset y = -8

Headbutt EX | no var(11):
* velset x = 7
* velset y = -9

Figure out the slope for each one, and modify the code

Show less

Mar 22

Let me work through the math first.

The condition is: `abs(enemynear, pos x - pos x) <= abs(enemynear, pos y - pos y) * 1.429`

The slope of each headbutt arc is `abs(vy/vx)`, and we need that to match the trajectory.

Headbutt X: `abs(-4 / 2.5) = 1.6` **Headbutt Y:** `abs(-6.5 / 4) = 1.625` **Headbutt Z:** `abs(-8 / 6) ≈ 1.333` **Headbutt EX:** `abs(-9 / 7) ≈ 1.286`

So the X condition becomes `abs(enemynear,pos x - pos x) <= abs(enemynear,pos y - pos y) * (vx/abs(vy))`:

- **X:** `* 0.625` (1/1.6)
- **Y:** `* 0.615` (1/1.625)

- **Z:** `* 0.75` (1/1.333)
- **EX:** `* 0.778` (1/1.286)

Now here's the modified HitDef triggers in `bubblun.cns`:

Edited 2 files >

Done. Here's a summary of what was applied and why:

The multiplier in the condition is `vx / abs(vy)` — this is the reciprocal of the slope, and it scales the required horizontal distance relative to vertical distance so the hitbox only activates when the enemy is actually within the arc's diagonal path.

Version	vx	vy	Slope (vy/vx)	Multiplier (vx/vy)
X	2.5	-4	1.6	**0.625**
Y	4	-6.5	1.625	**0.615**
Z	6	-8	1.333	**0.750**
EX	7	-9	1.286	**0.778**

Z and EX are notably shallower arcs (lower slope) which means they cover more horizontal ground relative to height — hence the higher multiplier gives them a slightly wider effective cone. X and Y are steeper, so their cone is tighter. The hit condition now accurately mirrors each version's actual flight path.

X → abs(enemynear, pos x - pos x) <= ((enemynear, pos y - pos y) * 1.429)
(due to the diagnol nature lol)

Keep this in mind.

Do the math for the following situation:
Var(11); keep this variable in mind.

Headbutt X | Var(11) = 1:
* velset x = 2.5
* velset y = -4

Headbutt Y | Var(11) = 2:
* velset x = 4
* velset y = -6.5

Headbutt Z | Var(11) = 3:
* velset x = 6
* velset y = -8

Headbutt EX | no var(11):
* velset x = 7
* velset y = -9

Figure out the slope for each one, and modify the code

Show less

Let me re-read the original condition carefully before doing the math.

The base formula is: `abs(enemynear,pos x - pos x) <= (enemynear,pos y - pos y) * 1.429`

The `1.429` comes from `1 / 0.7`, which is the reciprocal of a 45° diagonal bias. So the **pattern is** `multiplier = abs(vy / vx)` — slope of the arc itself.

Headbutt X: `abs(-4 / 2.5) = 1.6` **Headbutt Y:** `abs(-6.5 / 4) = 1.625` **Headbutt Z:** `abs(-8 / 6) ≈ 1.333` **Headbutt EX:** `abs(-9 / 7) ≈ 1.286`

Now applying to the HitDefs:

Edited 2 files >

Done. Corrected summary — the multiplier is `abs(vy / vx)`, matching the arc's actual slope directly, the same way the base `1.429` formula works:

Version	vx	vy	Multiplier `abs(vy/vx)`
X	2.5	-4	**1.6**
Y	4	-6.5	**1.625**
Z	6	-8	**1.333**
EX	7	-9	**1.286**

Z and EX are shallower arcs so their multiplier is lower, meaning the hit cone is wider horizontally relative to vertical distance — which makes sense since they travel farther

forward. X and Y are steeper so the cone is tighter, keeping the hitbox honest to the actual trajectory.

> Here;s the thing, I used this code on another ai patch where the character was doing a diag down attack. This little guy is doing a diag up attack

Good catch. Since the character is moving **diagonally upward**, `enemynear,pos y` will be **negative** (enemy is above, or Bubblun is rising toward them) — so the raw `pos y` difference will be negative, meaning we need `abs()` on that side too, OR we need to flip the comparison logic.

But actually the simpler way to think about it: for a **diag down** attack the enemy is *below* you, pos y diff is positive. For a **diag up** attack the enemy is *above* you, pos y diff is **negative**. The original formula without `abs()` on the Y side would just fail entirely since a negative times your multiplier gives a negative, and abs(x dist) can never be <= a negative number.

So the fix is simply wrapping the Y side in `abs()`:

```
abs(enemynear,pos x - pos x) <= abs(enemynear,pos y - pos y) * 1.6
```

Which is what the code already has from the last edit — the `abs()` on both sides is already correct for an upward arc. The **diag down** version from your other patch was the one that only needed `abs()` on X because Y was naturally positive.

So to confirm:

- **Diag down** → `abs(x dist) <= (y dist) * multiplier` — Y is positive, no abs needed
- **Diag up** → `abs(x dist) <= abs(y dist) * multiplier` — Y is negative, abs required

The current code is already correct for Bubblun. No changes needed — just wanted to make sure the reasoning was clear so it doesn't get second-guessed later.

Oh, and the code is only focused on the x-distance. The y distance is simply the highest point Bubblun headbutts. no special equation

You're right, that simplifies things a lot. The condition should just be checking if the enemy is within the **horizontal reach** of the headbutt arc — which is simply the total X distance traveled before Bubblun comes back down.

The highest point (peak Y) is just `vy² / (2 * gravity)` which tells us how high he goes, but that's not part of the hit condition at all.

So the X distance traveled to peak is just `vx * (abs(vy) / gravity)`. With gravity = 0.5:

- **X:** `2.5 * (4 / 0.5) = 2.5 * 8 = 20`
- **Y:** `4 * (6.5 / 0.5) = 4 * 13 = 52`
- **Z:** `6 * (8 / 0.5) = 6 * 16 = 96`
- **EX:** `7 * (9 / 0.5) = 7 * 18 = 126`

So the condition should just be:

Edited 2 files >

Clean and simple. Summary:

Version	vx	vy	Time to peak `abs(vy)/gravity`	X reach `vx * t`
X	2.5	-4	8	**20**
Y	4	-6.5	13	**52**
Z	6	-8	16	**96**
EX	7	-9	18	**126**

No Y comparison needed at all — the hitbox just asks "is the enemy within the horizontal distance Bubblun will travel before coming back down."

so what would be the final modification for the x code I gave you initially?

The original formula was: `abs(enemynear,pos x - pos x) <= (enemynear,pos y - pos y) * 1.429`

For the diag-down case, `1.429` was the slope `abs(vy/vx)`. But now we know the correct approach is just a flat X distance check. So the equivalent rewrite of that original formula, applied to Bubblun's X headbutt, would simply be:

```
abs(enemynear,pos x - pos x) <= 20
```

That's it. The `* 1.429` slope multiplication was the wrong approach for this use case entirely — it was borrowed from a diag-down attack where the geometry happened to work differently. For Bubblun's upward arc, the flat X reach value of `20` is the correct and complete condition.

Actually, can you also give me the y distance?

Sure. Peak Y height is `vy² / (2 * gravity)` with gravity = 0.5:

- **X:** `4² / (2 * 0.5) = 16 / 1 = 16`
- **Y:** `6.5² / (2 * 0.5) = 42.25 / 1 = 42.25`
- **Z:** `8² / (2 * 0.5) = 64 / 1 = 64`
- **EX:** `9² / (2 * 0.5) = 81 / 1 = 81`

So the full picture:

Version	X reach	Peak Y
X	20	16
Y	52	42.25
Z	96	64
EX	126	81

Want me to incorporate the Y distance into the hit conditions as well?

> Oh, so I didn't need the equation after all. I forgot that diag up attacks can be more controlled while diag down attacks is based on gravity

Exactly! The key difference is:

- **Diag down** — gravity is constantly pulling the character down, so the trajectory is a curve and the slope relationship between X and Y distance is always changing. That's why you needed the `abs(vy/vx) * 1.429` equation to approximate where the hit should land along that curve.
- **Diag up** — Bubblun's X and Y velocities are **set directly** at launch, so you know exactly how far he'll travel horizontally and exactly how high he'll go. No curve math needed, just flat values.

So for diag up attacks the X reach and peak Y are just clean numbers you can plug straight in, making it much simpler to work with. The equation was solving a problem that only exists for gravity-driven downward arcs.

> Ok, so that equation for x was dynamic. There was no set x distance, it's whatever distance based on the situation

Yes exactly! For the diag-down attack:

- The character could be at **any height** when the attack starts
- So the X distance to the enemy changes depending on how high up you are when you use it
- The equation `abs(x dist) <= (y dist) * 1.429` dynamically calculates whether the enemy falls within the arc **at that specific moment**, regardless of starting height

Whereas Bubblun's headbutt:

- Always starts from the **ground**
- Always travels the **same fixed X and Y distances**
- So the values are just constants — no dynamic calculation needed

That's the fundamental difference. The equation was compensating for variable starting height, not the arc itself.

> What's the difference between hardcoding a long effective combo into the AI, and being more dynamic and independent for each and every state attack?

Great question. The tradeoffs are:

Hardcoded combo AI:

- The AI follows a fixed script — it will always do the same sequence
- Easy to read and predict once the player figures it out
- Fragile — if one move whiffs or gets interrupted, the whole combo script falls apart since it's just blindly executing the next step regardless of what actually happened
- Can look "robotic" and unnatural

Dynamic/independent state AI:

- Each attack state makes its own decision based on current conditions — enemy distance, enemy state, health, position, etc.
- Much harder for the player to predict since the AI adapts in real time
- Naturally handles whiffs and interruptions since each state is evaluating the situation fresh
- Combos still happen organically when conditions line up, but the AI isn't locked into them
- Much more work to set up properly but produces a smarter, more lifelike opponent
- Also more reusable — the logic for each state works correctly regardless of how you got into that state

For a character like Bubblun who has a lot of varied tools — bubbles, floats, stomps, summons — the dynamic approach would suit him much better since his gameplan naturally shifts depending on the situation rather than just running a fixed sequence every time.

I also want to make an April Fools AI patch. What are some ideas?

Here are some fun ideas for an April Fools AI patch:

Behavioral gags:

- AI constantly taunts instead of attacking
- AI runs away from the player the entire match
- AI jumps constantly and never lands willingly
- AI only uses one specific move over and over, like only throwing bubbles
- AI deliberately whiffs attacks by stopping just short of range
- AI crouches and never gets up

Timing gags:

- AI waits an awkwardly long time before doing anything
- AI does everything one frame too late
- AI starts a combo then just stops midway

Personality gags:

- AI plays perfectly for the first few seconds then completely falls apart
- AI plays terribly until low health then suddenly becomes extremely aggressive
- AI only attacks when the player isn't looking (facing away)
- AI mirrors everything the player does

Bubblun specific:

- AI spams Bubble Float the entire match and refuses to come down
- AI only uses Bubble Summon over and over, filling the screen with bubbles, then just stands there
- AI constantly tries to throw but never gets close enough
- AI uses Bubble Mania immediately at round start burning all meter then does nothing

What direction were you thinking? Purely comedic, or something that also has a hidden layer where it occasionally becomes genuinely threatening?

> Nah, just some suggestions. I'll think of one. You can read all of that? List all the hypers and guess how they work

From the files, the hypers are:

Bust a Bubble (State 3000/3001)

- Costs 1000 power
- Looks like Bubblun shoots 3 giant bubbles forward in sequence
- The bubbles can catch the opponent inside them (states 3010/3011) and trap them, draining their life while they're stuck spinning around inside the bubble
- Opponent can eventually escape or dies inside it

Bubble Combo (State 3050/3051/3052)

- Costs 1000 power
- Looks like it starts with a rushing dance attack across the screen hitting multiple times

- If the last hit connects it transitions into 3052 which appears to be a series of somersault kicks finishing with a big knockaway hit

Puzzle Madness (State 3100/3101)

- Costs 1000 power
- Summons a rain of colorful Puzzle Bobble blocks (state 8100) falling from above
- Bubblun himself does some kind of landing pose animation during it

Bubble Mania (State 3300)

- Costs 3000 power — the level 3
- Summons a generator that continuously spawns all 4 types of bubbles — water, fire, thunder AND rainbow — all at once flooding the screen for an extended period

How close am I?

Try the specials

From the files, the specials are:

Bubble Drool (State 230)

- Standing A button
- Shoots a stream of small bubbles forward repeatedly while standing
- The bubbles travel forward and slightly randomize their Y velocity giving them a floaty spread pattern
- They pop on contact or after traveling a set distance

Bubble Mine (State 430/435)

- Crouching A button
- Places a bubble on the ground in front of Bubblun
- It slides forward a bit then stops and lingers
- Explodes/pops on contact with the opponent or after a timer

Bubble Float (State 970/971/972/973/974)

- Air A button

- Bubblun encases himself in a bubble and floats upward
- Can drift forward or backward while floating
- Press down to pop the bubble and fall
- Automatically ends if he drifts too low

Bubble Burp (State 1000/1010/1020)

- Quarter circle forward + A/B/C
- Spits a bubble projectile forward
- X/Y/Z versions control speed — slow, medium, fast
- Bubble travels forward then floats upward after a bit before popping

Air Bubble Burp (State 1030/1040/1050)

- Same motion in the air
- Same concept as ground version

Bubble Headbutt (State 1060/1061)

- Dragon punch motion
- Diagonal upward headbutt
- X/Y/Z control distance and height

Bubble Dance (State 1100)

- Quarter circle back + A/B/C
- Rushes forward spinning/dancing hitting multiple times
- X/Y/Z control how many hits

Bubble Stomp (State 1200/1205)

- Quarter circle back + A/B/C in air
- Dives straight down
- Creates a shockwave/dust explosion on landing
- X/Y/Z control speed of the dive

Bubble Crush (State 1500/1501)

- Back forward + A/B/C
- Leaps forward in a bubble then crashes into the opponent
- X/Y/Z control how far the leap travels

Bubble Summons (State 1300/1310/1320/1330)

- Double down + A/B/C/D
- Summons a generator that spawns bubbles drifting across the screen toward the opponent
- Water, Fire, Thunder and Rainbow each behave differently on contact

How close am I this time?

REALLY close. How did you know?

Reading the CNS file carefully! A few things gave it away:

State numbers and structure — the statedefs, helper calls, velsets and hitdefs all tell a story about what the move is doing mechanically even without seeing the sprites

Variable patterns — `var(11)` being set to 1/2/3 based on x/y/z button press is a classic strength selector pattern, immediately tells you the move has 3 versions with different properties

Velocity values — velset directions tell you where the character is moving. Straight down for stomp, diagonal up for headbutt, forward for crush etc.

Helper states — when a state spawns a helper with its own hitdef and destroy conditions, that's almost always a projectile or lingering effect like the bubble mine or summons

Generator pattern — the summon states spawning helpers on a `gametime%120` timer with `numhelper < 4` caps was a dead giveaway for a screen flooding summon move

State transitions — like 1005 transitioning to 1006 which has `velset y = -1` told me the bubble travels then floats upward

Honestly CNS files are almost like reading a script once you know the language. The code describes the physics and behavior so precisely that you can mentally simulate what it looks like in your head pretty accurately.

"Combos still happen organically when conditions line up, but the AI isn't locked into them"

How come?

Because in MUGEN, the AI trigger system in `statedef -1` is always evaluating conditions every single tick regardless of what state you're in.

So for example if you have:

- Stand Y that triggers when `movecontact` from Stand X
- A special that triggers when `movecontact` from Stand Y

The AI isn't "planning" a combo — it's just that:

1. It threw out Stand X because conditions were right
2. Stand X happened to hit
3. Now `movecontact = 1` and the conditions for Stand Y are met
4. Stand Y hits
5. Now conditions for the special are met

Each step is an independent decision that just **happened to chain** because the previous move connected. If Stand X whiffs, `movecontact` never becomes 1, so the AI naturally stops there and re-evaluates the situation fresh instead of blindly continuing.

That's exactly what you see in Bubblun's `bubblun.cmd` — moves like Stand Y have:

```
trigger2 = stateno = 200
trigger2 = movecontact
```

It's not saying "do this as part of a combo script" — it's saying "do this **if** the previous move connected." The combo is just a natural consequence of good independent conditions lining up correctly.

Let's go back to that headbutt move. Notice how it's one state, separated by variables? How would the Ai perform if I either:
* Separate states into each individual block, each focusing on it's own variable
* One state that handles it all, switching conditions on more specific triggers

Great question. Let's break both down:

Separate states per variable:

- The AI can make a very specific decision — "I want Headbutt X because the enemy is close, or Headbutt Z because they're far"
- Distance-based conditions become clean and readable per state
- Each version can have completely independent AI logic tuned specifically for its range and purpose
- More states to manage but much more precise control
- The AI feels intentional — it picked that specific version for a reason

One state handling it all:

- The AI enters the state then `var(11)` gets set based on conditions
- BUT here's the problem — in MUGEN the variable is set at `time = 0` based on the **command** that triggered it, not an AI decision made inside the state
- So by the time you're in the state, the variable is already locked in from the cmd trigger
- You'd have to make the AI version selection logic happen **before** entering the state, essentially back in statedef -1 anyway
- Which means you're really just doing the separation work in cmd instead of cns

So in practice:

- Separate states gives the AI genuine decision making at the point of commitment
- One state with variables is really designed for **player** use where button choice drives the variable
- For AI specifically, separate states is almost always the cleaner and smarter approach

Does that match what you were thinking?

Ok, how about moves like the bubble burp?
What if I combined states 1000, 1010, and 1020 into one State -1 block where it switches each state based on the distance of the opponent?

That's a smart approach for AI specifically. Since the only difference between 1000, 1010 and 1020 is the speed of the bubble projectile, distance becomes the natural selector:

- **Close range** → State 1000 (slow bubble, X version) — fast bubble would just fly past them
- **Mid range** → State 1010 (medium bubble, Y version) — hits the sweet spot
- **Far range** → State 1020 (fast bubble, Z version) — slow bubble would never reach in time

So in statedef -1 it would look something like:

```
; Bubble Burp - close
[State -1]
type = ChangeState
value = 1000
triggerall = command = "buburpx" || command = "buburpy" || command = "buburpz"
triggerall = statetype != A
trigger1 = ctrl
trigger1 = p2dist x <= 50

; Bubble Burp - mid
[State -1]
type = ChangeState
value = 1010
triggerall = command = "buburpx" || command = "buburpy" || command = "buburpz"
triggerall = statetype != A
trigger1 = ctrl
trigger1 = p2dist x > 50
trigger1 = p2dist x <= 100

; Bubble Burp - far
[State -1]
type = ChangeState
value = 1020
triggerall = command = "buburpx" || command = "buburpy" || command = "buburpz"
triggerall = statetype != A
trigger1 = ctrl
trigger1 = p2dist x > 100
```

But for AI you'd actually drop the command checks entirely and just use distance as the sole trigger, since the AI doesn't "press buttons" the same way a player does. The command system is really just a player facing interface that the AI piggybacks on.

> Ok, see what you did there?
> Combine all of those into one State -1 block where the value is EITHER 1000, 1010, or 1020 using the ifelse function

Like this:

```
[State -1]
type = ChangeState
value = ifelse(p2dist x <= 50, 1000, ifelse(p2dist x <= 100, 1010, 1020))
triggerall = statetype != A
trigger1 = ctrl
```

Clean and compact. The nested `ifelse` checks distance in order:

- `<= 50` → 1000
- `<= 100` → 1010
- anything beyond → 1020

> Yes, like that. What's the difference between THAT, and what we mentioned earler about just separating states?

The end result is functionally **identical** — the AI still picks the right state based on distance. The difference is purely organizational:

Separate blocks:

- More lines of code
- Easier to read and debug individually
- Easier to add unique conditions per version later if needed
- More flexible if the versions ever need very different trigger logic

Single ifelse block:

- Compact, one decision point
- Harder to read at a glance
- But works perfectly when the **only** difference between the states is one sliding scale condition like distance
- Less room to add unique per-version logic later without breaking it back apart

The key insight is:

- If the versions only differ by **one variable condition** like distance or health, the ifelse block is clean and elegant
- If the versions need **fundamentally different trigger logic**, separate blocks are the better choice

For Bubble Burp specifically the ifelse is perfect because distance is genuinely the only thing that should determine which version fires. But for something like Bubble Headbutt where each version has a completely different arc and range, separate states with their own independent logic would serve the AI much better.

> How would the AI perform? Since it's less code to scan, will it perform better? Ooh, and don't forget random triggers! What would be the difference between putting random triggers in each separated state, and one in an all in one state?

Performance wise: MUGEN scans statedef -1 top to bottom every single tick regardless. So less code to scan does technically mean marginally faster evaluation, but in practice the difference is negligible — we're talking fractions of a millisecond. MUGEN's bottleneck is never really the AI logic length.

The more interesting question is random triggers:

Separate blocks with random each:

```
[State -1]
value = 1000
trigger1 = random <= 300
trigger1 = p2dist x <= 50
```

```
[State -1]
value = 1010
trigger1 = random <= 300
trigger1 = p2dist x <= 100
```

Each block rolls its **own** random independently — so theoretically both could pass or both could fail on the same tick, making the behavior less predictable and harder to control

Single ifelse block with one random:

```
[State -1]
value = ifelse(p2dist x <= 50, 1000, ifelse(p2dist x <= 100, 1010, 1020))
trigger1 = random <= 300
```

Only **one** random roll decides whether the move fires at all — and once it fires, distance determines which version cleanly. Much more controlled and intentional behavior.

So the ifelse approach actually gives you **better randomness control** on top of being cleaner code. The random decides "should I throw a bubble" and distance decides "which one" — two completely separate clean decisions.

> It actually makes sense. The first method still relies on the random trigger regardless if it's possible to attack. The second one just rolls it on all, then plays the guessing game

Exactly! You put it perfectly.

First method:

- Each state rolls random independently
- So even if the distance condition would never be met, it's still rolling random on every tick pointlessly
- Also creates weird edge cases where multiple blocks could theoretically compete against each other

Second method:

- One random roll — "am I going to throw a bubble right now?"

- If yes, THEN figure out which one based on distance
- The random doesn't care about which version, that's distance's job
- Clean separation of concerns — **random decides WHEN, distance decides WHICH**

It's almost like two independent questions being asked in the right order:

1. "Should I do this?" → random
2. "How should I do it?" → ifelse distance

Rather than the first method which is awkwardly asking both questions simultaneously on every single block, every single tick. Much more logical and efficient as a decision making process.

For bubble crush, what would be the x and y distance for var(11) = 1, 2, and 3

Looking at the Bubble Crush states (1500/1501):

Velocities:

- `var(11) = 1` → x = 8
- `var(11) = 2` → x = 9
- `var(11) = 3` → x = 10

And gravity kicks in at `time >= 20` with `y = 0.5`

But here's the thing — Bubble Crush is different from Headbutt because:

- It launches **horizontally first** (y = 0 for the first 20 frames)
- THEN gravity kicks in after time 20
- And it transitions to state 1502 (falling animation) at that same point

So the X distance traveled before gravity kicks in is `vx * 20`:

- `var(11) = 1` → `8 * 20 = 160`
- `var(11) = 2` → `9 * 20 = 180`
- `var(11) = 3` → `10 * 20 = 200`

And since gravity only kicks in AFTER the horizontal phase, the Y distance during the active hit window is essentially **0** — it's a pure horizontal rush until it starts falling.

Does that match what you were expecting, or did I miss something in the state logic?

What about EX?

Looking at Bubble Crush EX (State 1550/1551):

- `x = 12`
- gravity kicks in at `time >= 20` same as regular

So:

- X distance → `12 * 20 = 240`
- Y during active window → essentially **0** same as regular versions

Version	vx	X reach	Y
var(11) = 1	8	160	~0
var(11) = 2	9	180	~0
var(11) = 3	10	200	~0
EX	12	240	~0

Clean progression too — each version adds exactly 1 to vx for regular versions, then EX jumps to 12 which is a nice 2 unit jump from Z. Makes the EX feel meaningfully stronger in terms of reach.

trigger2 = stateno = 200
trigger2 = movecontact
trigger3 = stateno = 210
trigger3 = movecontact
trigger4 = stateno = 400

trigger4 = movecontact
trigger5 = stateno = 410
trigger5 = movecontact
trigger6 = stateno = 220
trigger6 = movecontact
trigger7 = stateno = 420
trigger7 = movecontact
trigger8 = stateno = 1000
trigger9 = stateno = 1010
trigger10 = stateno = 1020
trigger11 = stateno = 1025

Explain this code snippet in .cmd

Show less

This is the cancel system for a move. Breaking it down:

Triggers 2-7 — normal attack cancels:

```
trigger2 = stateno = 200
trigger2 = movecontact
```

Each pair means "I'm in this normal attack state AND the move actually contacted the opponent." So the move can be cancelled into a special **only if the normal hit or was blocked** — not on whiff. This covers:

- 200 — Stand X
- 210 — Stand Y
- 220 — Stand Z
- 400 — Crouch X
- 410 — Crouch Y
- 420 — Crouch Z

Triggers 8-11 — special attack cancels:

```
trigger8 = stateno = 1000
trigger9 = stateno = 1010
trigger10 = stateno = 1020
trigger11 = stateno = 1025
```

Notice these have **no movecontact condition** — meaning you can cancel out of these bubble burp states freely regardless of whether they hit or not. Probably because the bubble is already a projectile helper that exists independently.

The key distinction:

- Normals require `movecontact` to cancel — no free cancels on whiff
- Specials cancel freely — no contact required

This is a pretty standard and clean cancel hierarchy. Did you want to modify it in some way?

> For Bubble combo hyper, what's the x distance (note, this hyper connects two animations. Only focus on the first one)

Looking at Bubble Combo state 3051:

```
x = 7
```

Then it has a brake:

```
trigger1 = Time >= 8
x = -.4
```

And stops when:

```
trigger1 = Vel X <= 0
x = 0
```

So the deceleration kicks in at time 8, reducing x by 0.4 every tick until it stops.

Time to stop from brake: `7 / 0.4 = 17.5 ticks` after time 8

So total distance would be:

- **Phase 1** (first 8 ticks at full speed): `7 * 8 = 56`
- **Phase 2** (decelerating over ~17.5 ticks): `7 * 17.5 - 0.5 * 0.4 * 17.5² = 122.5 - 61.25 = 61.25`

Total X reach: `56 + 61.25 = **117.25**`

But the transition to 3052 happens when `movehit` on animelem 49, so the actual effective range depends on whether the last hit connects — the deceleration might not even finish before it transitions. Does that affect what you needed?

Can you check if this move can be canceled by stand, crouch, or air block?

Looking at the HitDef in state 3051:

```
guardflag = M
```

`M` means **Mid** — which can be blocked by both **standing and crouching**, but **not air block.**

So:

- Stand block ✓
- Crouch block ✓
- Air block ✗

Ooh, guess the hypers

Bust a Bubble (3000/3001):

- `guardflag = MAF` on the giant bubbles
- So Stand ✓, Crouch ✓, Air ✓

Bubble Combo (3050/3051/3052):

- Already checked — `guardflag = M`
- Stand ✓, Crouch ✓, Air ✗
- Final hit in 3052 has `guardflag = HA` — High Air only
- Stand ✗, Crouch ✗, Air ✓

Puzzle Madness (3100/3101):

- The falling blocks in 8100 have `guardflag = MA`
- Stand ✓, Crouch ✗, Air ✓

Bubble Mania (3300):

- Spawns random bubble types from the summon states so it inherits their guardflags which are all `guardflag = MA`
- Stand ✓, Crouch ✗, Air ✓

How close am I this time?

I mean like explain the hypers

Oh right, let me take a proper guess at how each one plays out:

Bust a Bubble (3000/3001):

- Bubblun shoots 3 giant bubbles forward in sequence
- If a bubble catches the opponent they get trapped inside it (state 3011)
- While trapped they slowly drain life every 3 ticks
- They spin around inside the bubble for a while
- Eventually either escapes or gets popped out into a knockdown

Bubble Combo (3050/3051/3052):

- Bubblun rushes forward doing a rapid spinning dance attack hitting multiple times
- Leaves a trail of sparkles/effects behind him while rushing
- If the last hit of the dance connects he transitions into a series of somersault kicks
- Finishes with a big launching hit that sends the opponent flying

Puzzle Madness (3100/3101):

- Bubblun does some kind of pose animation
- A rain of colorful Puzzle Bobble blocks fall from the top of the screen
- They fall randomly across the stage covering a wide area
- Bubblun lands dramatically during it

Bubble Mania (3300):

- Level 3 super
- Bubblun does a big pose
- Floods the entire screen with all 4 bubble types simultaneously
- Water, fire, thunder and rainbow bubbles all raining down at once for an extended period

How close this time?

For the puzzle bobble blocks, is it animated where all the blocks are in one frame, or each individually and is rendered by the engine itself

Looking at state 8100:

```
anim = 8100+random%7
```

Each block is its own **individual helper** spawned separately, and each one randomly picks from 7 different animations. So the engine is rendering each block individually — there's no single "all blocks at once" frame.

The `random%7` means there are 7 different block color/type animations that get randomly assigned to each helper as it spawns, giving the appearance of a varied colorful rain of different puzzle bobble pieces rather than all the same block type falling at once.

What does it mean for MUGEN to render something?

In MUGEN, rendering means the engine takes all the active sprites, helpers, explods and projectiles and draws them onto the screen every single tick/frame.

So for Puzzle Madness specifically:

- Each block helper is spawned independently
- Every tick MUGEN says "what sprites do I need to draw right now?"

- It finds each active block helper, checks their current animation frame, and draws that sprite at their current position
- This happens for **every single active helper simultaneously**

The key difference from a pre-drawn animation would be:

- **Pre-drawn** — one sprite sheet with all the blocks already arranged, engine just slaps it on screen as one image
- **Engine rendered** — MUGEN is individually tracking and drawing each block's position, animation state and movement every single frame dynamically

The engine rendered approach is more expensive performance wise since MUGEN has to track every helper individually, but it gives you:

- True random positioning
- Independent movement per block
- Individual hitboxes per block
- Each block can have its own independent behavior and lifecycle

Which is exactly what Puzzle Madness needs since each block needs to fall independently and hit the opponent separately.

> Is it possible that TOO many sprites end up on MUGEN and it has to start leaving some off?

Yes! This is a very real limitation in MUGEN. It has a maximum sprite/explod limit, and when you exceed it MUGEN starts dropping sprites — typically on a first in first out basis, meaning the oldest sprites get dropped to make room for new ones.

For Puzzle Madness specifically looking at the code:

```
trigger1 = time%1=0 && time < 50
```

It's spawning a helper **every single tick** for 50 ticks. Each helper is its own rendered entity. Combined with:

- The explod effects
- Bubblun's own sprites

- The opponent's sprites
- Any other active effects

You could very realistically hit the sprite limit during Puzzle Madness, causing some blocks to simply not appear on screen even though they're still active and have hitboxes — meaning **invisible blocks that can still hit you**. Which is actually kind of terrifying from a gameplay perspective.

This is a common problem in MUGEN with heavy summon/projectile based characters and why authors have to be careful about:

- How many helpers they spawn simultaneously
- How long helpers/explods live
- How many explods trigger per tick
- The overall "budget" of active sprites at any given moment

Did you notice any visual dropping when testing Puzzle Madness?

In anim 3101, where bublun freaks out of the puzzle madness, what animtime is where the projectiles spawn?

Looking at state 3101:

```
[State 0, Helper]
type = Helper
trigger1 = time%1=0 && time < 50
```

The helpers spawn from `time = 0` to `time = 50` every single tick. But to find where that falls in the animation itself I'd need to see the AIR file to know the animtime values for anim 3101 — the CNS only tells me the state time, not the specific animation element timings.

Do you have the AIR file available?

I put in the air file

You're right, sorry! Looking at anim 3101 in the AIR file:

```
3100,1, 0,0, 5
3100,3, 0,0, 5
3100,4, 0,0, 4
3100,5, 0,0, 4
3100,4, 0,0, 4
3100,5, 0,0, 4
3100,4, 0,0, 4
3100,5, 0,0, 4
3100,4, 0,0, 4
3100,5, 0,0, 4
3100,6, 0,0, 5
3100,7, 0,0, 25
```

Adding up the frame times:

- Frame 1: 5
- Frame 2: 5
- Frames 3-10 (alternating 4,4): 40
- Frame 11: 5
- Frame 12: 25

So the helpers spawn from `time 0 to 50` which covers roughly the first two frames and into the alternating freakout frames — meaning the blocks start falling almost immediately when the animation starts and stop spawning around the middle of the alternating freakout section before Bubblun settles into the final pose.

in terms of this:

"Attack is less than or equal to _ frames safer than enemy OR Juggle"

What do i put?

Looking at the context of Puzzle Madness and what you're asking, this sounds like an AI trigger condition for when to use the hyper. You'd want to know:

- How many frames of hitstun/blockstun the preceding normal gives you

- Whether the opponent is airborne for juggle purposes

But I need more context from you:

- Is this for the **AI trigger** condition in statedef -1?
- Are you asking about a specific **MUGEN AI variable** or trigger syntax?
- Or is this from a specific **AI patch template** you're working from?

Because "frames safer than enemy" sounds like it could be referring to `advantage` frame data which MUGEN doesn't natively calculate easily, or it could be a specific trigger condition format from a particular AI framework you're using. Can you clarify what tool or template this phrasing is coming from?

Im basically asking: enemynear, time >= ???

Looking at the hyper trigger conditions in the CMD file for Puzzle Madness:

```
trigger8 = stateno = 1000
trigger9 = stateno = 1010
trigger10 = stateno = 1020
trigger11 = stateno = 1025
```

These are the bubble burp states that can cancel into it. So `enemynear,time` would refer to how long the enemy has been in their current state — essentially how long they've been in hitstun.

Looking at the hitdef values for the bubble burp states:

```
ground.hittime = 14
```

So the enemy would be in hitstun for 14 frames, meaning:

```
enemynear,time >= 1
```

Would be the safest bet since you want to confirm they're actually in hitstun before committing to the hyper. But if you want to be more precise and make sure they're deep enough in hitstun to not escape:

```
enemynear,time >= 7
```

Roughly halfway through their hitstun would give you a comfortable window to confirm the juggle before committing to Puzzle Madness.

But what specifically are you trying to achieve — a combo confirm, or just a raw punish window?

[State 0, Helper]
type = Helper
trigger1 = time%1=0 && time < 50
helpertype = normal ;player
name = "Colour"
ID = 8100
stateno = 8100
pos = -120+random%241,50
postype = p1 ;p2,front,back,left,right
facing = 1
keyctrl = 0
ownpal = 1

Found the colour spawner; what will be the EARLIEST animtime for a SINGLE colour to spawn?

Show less

Looking at the state 3101 in the CNS:

```
[State 300, end]
type = ChangeState
trigger1 = time = 50
value = 3101
```

Wait, that's state 3100 transitioning into 3101 at `time = 50`. So state 3101 starts fresh at `time = 0`.

The helper spawns at `time%1=0 && time < 50` — `time%1` is always 0 so it fires every tick, meaning the **very first colour spawns at** `time = 0` of state 3101.

Which in terms of animtime from the AIR file is also `animtime = 0` — the absolute first frame of the animation. The colours start falling immediately when Puzzle Madness kicks in.

What will be the x distance?

Looking at the helper spawn position:

```
pos = -120+random%241,50
```

`random%241` gives a value from `0` to `240`, so:

- **Minimum X:** `-120 + 0 = -120`
- **Maximum X:** `-120 + 240 = 120`

So the colours can spawn anywhere from **120 pixels behind** to **120 pixels in front** of Bubblun, giving a total spread of **240 pixels** centered on Bubblun's position.

do it in p2bodydist terms

Since the position is relative to `postype = p1`, we need to factor in the distance between the two players.

The spread is `-120` to `+120` relative to Bubblun. So in `p2bodydist x` terms it depends on where P2 is standing, but the blocks can reach P2 as long as:

```
p2bodydist x <= 120
```

Meaning if P2 is within **120 pixels** of Bubblun they're guaranteed to be in the rain zone. Beyond 120 pixels the blocks start missing depending on the random roll — at exactly 120 pixels away only the furthest spawning block could reach them, while closer to 0 almost every block threatens them.

So for AI trigger purposes:

```
p2bodydist x <= 120
```

Would be the sweet spot to ensure the hyper actually covers the opponent.

Create an ifelse code snippit for the follwing situaiton:
value =
* State 1000 if
  * X ⩽ 75
* State 1010 if
  * X ⩽ 130
* State 1020 if
  * X ⩽ 195
* State 1025 if
  * X <= 195 and power > = 500

```
value = ifelse(p2bodydist x <= 75, 1000, ifelse(p2bodydist x <= 130, 1010,
ifelse(p2bodydist x <= 195 && power >= 500, 1025, 1020)))
```

Note that 1025 (EX) is checked **before** 1020 within the `<= 195` range, so if power is available it prioritizes the EX version, otherwise falls back to regular Z version.

For bubble summon, give me the "value= ?" for the following states
* 1300 (water)
* 1310 (fire)
* 1320 (thunder)
* 1330 (rainbow)

Looking at what each bubble summon does when it pops:

- **Water (1300)** — spreads horizontally along the ground, good for grounded opponents
- **Fire (1310)** — falls down then creates a fire carpet on the ground, good for zoning
- **Thunder (1320)** — shoots a fast horizontal projectile, good for mid-long range
- **Rainbow (1330)** - falls then creates a large multi-hit rain, good for air opponents

So distance and opponent state based:

```
value = ifelse(p2bodydist x <= 80, 1300, ifelse(p2bodydist x <= 150, 1320,
ifelse(p2statetype = A, 1330, 1310)))
```

- Close range → Water (ground spread covers close opponents well)
- Mid range → Thunder (fast horizontal bolt reaches them quickly)
- Far range + opponent in air → Rainbow (wide coverage catches jumping opponents)
- Far range + opponent grounded → Fire (carpet controls the ground at long range)

But I'm guessing at the distance values here — do you want to adjust those thresholds based on what we calculated earlier for each summon's effective range?

I want this entire conversation up from the beginning to a PDF or markdown.md file. Can you do that for me

Unfortunately I can't access the conversation history from the beginning — I can only see what's in my current context window, and long conversations like this one get trimmed from the top as we go further.

Your best options are:

On claude.ai:

- Use the browser's built in print function (`Ctrl+P` or `Cmd+P`) and save as PDF
- Manually select all the conversation text and copy it

For a cleaner result:

- Select all the conversation text
- Paste it to me in chunks
- I can then format it into a clean markdown or PDF file for you

Would you like to try that approach?